

Protocol-Driven Development: Governing Generated Software Through Invariants and Continuous Evidence

Jun He, Deying Yu

OpenKedge.io
junhe@openkedge.io, deying@openkedge.io

Abstract

Automated program synthesis lowers the cost of producing implementations but introduces a harder governance problem: determining which generated artifacts are admissible. Natural-language specifications are ambiguous, and example-based tests sample only part of the behavioral space. Used alone, neither provides a sufficient control boundary.

We introduce **Protocol-Driven Development (PDD)**, where the primary software artifact is a machine-enforceable protocol rather than code. We define a protocol as the triplet $\mathcal{P} = (\mathcal{S}, \mathcal{B}, \mathcal{O})$, specifying structural, behavioral, and operational invariants. Their conjunction defines the admissible implementation space of a software component.

Under PDD, implementations are replaceable realizations discovered through constrained search. An implementation is admitted only if it satisfies the protocol and produces a verifiable *Evidence Chain* of compliance. Admission is grounded in protocol satisfaction and recorded evidence rather than trust in the generator.

For deployed systems, we extend the Evidence Chain into a *Dynamic Evidence Ledger*. Runtime verifiers append signed observations, invariant checks, and violations to the ledger, allowing monitorable obligations to be continuously attested. This connects live failures back to the generation loop without granting the generator runtime authority.

Combining formal methods, property testing, runtime verification, policy-as-code, and software provenance, PDD defines a governance model for automated software engineering. Its organizing principle is that code is transient, while the protocol carries durable authority.

Introduction

Generative program synthesis has reduced the marginal cost of producing candidate implementations. Boilerplate, interface implementations, service scaffolds, and many routine code changes can now be produced quickly by synthesis engines. This shift moves the central pressure from production to admission, and from admission to continued accountability in operation. The difficult question is no longer only how to produce code, but how to decide which generated implementations are structurally valid, behaviorally correct, operationally bounded, eligible for regeneration, and still compliant after deployment.

Traditional software engineering methodologies were developed when implementation effort dominated many work-

flows. In *Spec-Driven Development (SDD)*, natural-language documents describe intended structure and behavior, but they are often too ambiguous for automated admission. Different engineers—or different generative systems—may interpret the same requirement incompatibly, producing semantic drift, inconsistent assumptions, and hidden side effects.

Test-Driven Development (TDD) improves on this model by encoding expected behavior in executable tests (Beck 2003). Tests reduce ambiguity and increase confidence in functional behavior, but they are fundamentally extensional: they verify selected cases rather than defining the full space of admissible implementations. They rarely express structural contracts, authority boundaries, or operational constraints such as latency limits, side-effect restrictions, dependency controls, or resource quotas.

Automated code generation makes these weaknesses harder to ignore. When implementations can be synthesized and regenerated at low cost, source code becomes less durable than the constraints that determine whether the code is admissible. The operative question changes from “How do we implement this component?” to “What constraints must every acceptable implementation satisfy?” In this setting, software engineering becomes less a matter of prescribing one implementation and more a matter of defining the invariant boundary within which implementations may be safely discovered.

We call this model **Protocol-Driven Development (PDD)**. In PDD, the primary artifact is a machine-enforceable protocol rather than implementation code. We formally define a protocol as

$$\mathcal{P} = (\mathcal{S}, \mathcal{B}, \mathcal{O}),$$

where $\mathcal{S}$ denotes *structural invariants*, $\mathcal{B}$ denotes *behavioral invariants*, and $\mathcal{O}$ denotes *operational invariants*. Structural invariants define rigid type and interface contracts through typed handshakes. Behavioral invariants specify semantic properties that must hold for all admissible implementations. Operational invariants impose explicit capability boundaries covering side effects, latency, external dependencies, and resource consumption. Their conjunction defines the admissible implementation space within which automated generators may explore.

Under PDD, implementations are treated as replaceable realizations of a governing protocol. Automated generators may generate, discard, and regenerate candidate code, but a

candidate is admissible only if it satisfies the protocol. We propose a *Validator Loop* that performs structural validation, property-based verification, and operational conformance checks before admission. Admitted implementations then produce an *Evidence Chain*: a cryptographically verifiable record of the governing protocol, implementation metadata, validation results, and observed behavioral characteristics. The Evidence Chain supports auditability, accountability, and replay of admission decisions when replay inputs are preserved.

Pre-deployment evidence is not the end of governance. Production traffic can expose memory leaks, race conditions, dependency drift, adversarial inputs, and operational degradation that were absent from simulation. PDD therefore extends Evidence Chains into *Dynamic Evidence Ledgers*: append-only records that include signed runtime observations, invariant checks, violations, and remediation outcomes. Runtime verification layers enforce protocol boundaries outside the generated implementation, and violation blocks can be converted into structured repair contexts for the next generation attempt.

For concreteness, Appendix sketches a minimal protocol bundle, typed handshake, behavioral invariant set, capability manifest, and evidence record.

We term the recurring cost of this ambiguity the *Natural Language Tax*: the cost of interpreting, reconciling, and maintaining ambiguous textual descriptions. Rather than describing desired behavior in prose and relying on downstream interpretation, PDD encodes invariants directly as machine-enforceable assertions. Typed handshakes address type drift, property-based assertions address intent drift, and capability manifests address side-effect drift. The protocol becomes the authoritative admission artifact, and valid implementations are constrained to remain within its boundaries.

The model draws on formal methods, property-based testing, zero-trust security, and generative systems (Hoare 1969; Clarke, Grumberg, and Peled 1999; Claessen and Hughes 2000; Rose et al. 2020). It combines invariants, executable properties, explicit verification, and constrained search into a development model for software synthesis.

PDD does not claim to invent interface schemas, property-based testing, policy enforcement, or provenance and attestation mechanisms. Its novelty is the elevation of the protocol to the primary development artifact: a single governing object that jointly specifies structure, behavior, operational authority, and evidence-producing admission for generated software. Schema-first or interface-first development stabilizes component boundaries but usually leaves behavior, authority, and admission evidence outside the interface artifact. TDD and property-based testing constrain selected behavioral properties but do not by themselves define operational capability boundaries or provenance-linked admission. Policy-as-code governs authority but is typically external to software construction; supply-chain provenance records where artifacts came from rather than defining which artifacts are admissible. PDD unifies these concerns into a protocol-centered development model in which generated implementations are proposals admitted only through invariant satisfaction and verifiable evidence.

PDD also relates to runtime governance for autonomous systems while remaining independent of any particular deployment stack. Sovereign Agentic Loops (He and Yu 2026b) and OpenKedge (He and Yu 2026a) separate model-generated proposals from privileged execution through control boundaries, policy evaluation, and evidence records. PDD applies the same proposal-admission structure to software construction and then carries it into operation through runtime attestation.

The thesis of this work is:

Code is transient; protocol is sovereign.

By making protocols the durable engineering artifact, PDD treats software creation as constrained discovery: implementations may change while admissibility conditions remain explicit, testable, and auditable.

Contributions. The paper makes the following contributions:

1. We propose Protocol-Driven Development (PDD), a model that treats protocols, rather than implementation code, as the durable artifact of automated software engineering.
2. We formalize a protocol as a triplet of structural, behavioral, and operational invariants that together define an admissible implementation space for generated software.
3. We define protocol compliance, protocol-level substitutability, and evidence-producing acceptance as the core relations governing admission under PDD.
4. We outline the Validator Loop, an architectural pattern that separates protocol authoring, implementation generation, validation, and evidence preservation.
5. We introduce Evidence Chains and Discovery Logs as artifacts for linking candidate implementations to validation outcomes, provenance, admission decisions, and later runtime observations.
6. We extend Evidence Chains into Dynamic Evidence Ledgers, define continuous protocol attestation, and introduce a Runtime Verification Layer for enforcing invariants after deployment.
7. We specify a closed-loop remediation path in which signed runtime violation blocks become repair context for subsequent generation and validation.
8. We characterize the Natural Language Tax and argue that machine-enforceable protocols provide a disciplined way to move from narrative intent to explicit admissibility conditions.
9. We synthesize connections to formal methods, property-based testing, runtime verification, policy-as-code, software provenance, automated software engineering, and runtime governance for autonomous systems.

Background and Motivation

Protocol-Driven Development (PDD) sits at the intersection of specification, testing, formal verification, policy enforcement, and automated program synthesis. Each tradition addresses part of the correctness problem, but none by itself provides an admission model for generated implementations.

Spec-Driven Development

Spec-Driven Development (SDD) treats the specification as the prior artifact that guides implementation. Specifications range from prose requirements to machine-readable interface descriptions such as OpenAPI, JSON Schema, and Protocol Buffers (OpenAPI Initiative 2021; JSON Schema 2022; Google 2008). These artifacts establish a shared vocabulary and support automation such as documentation, client stubs, scaffolds, validation, and interface conformance testing.

SDD, however, leaves a semantic gap between what is written and what must be enforced. A requirement such as “return the current user profile” rarely specifies determinism, retry behavior, external calls, or latency budgets. Schema languages reduce structural ambiguity, but they usually do not capture full behavioral and operational semantics. PDD begins from this gap: the artifact that governs generated software must be more restrictive than prose and more expressive than schema alone.

Test-Driven Development

Test-Driven Development (TDD) moves part of engineering intent into executable form. Tests expose assumptions, define concrete acceptance criteria, and reject regressions automatically (Beck 2003). Fuzzing and property-based testing extend this idea by exploring larger behavior spaces through generated inputs (Claessen and Hughes 2000).

Yet tests remain partial witnesses. They may explore examples or input families, but they do not necessarily define the full protocol by which a module is permitted to exist inside a larger system. Tests also tend to focus on functional behavior while leaving authority boundaries implicit. In PDD, tests are one mechanism for validating behavioral invariants, not the complete engineering artifact.

Formal Methods

Formal methods provide an important precedent for PDD. Hoare logic, model checking, TLA+, and Alloy show that invariants, preconditions, postconditions, temporal properties, and relational constraints can expose design errors before implementation (Hoare 1969; Clarke, Grumberg, and Peled 1999; Lamport 2002; Jackson 2002).

PDD does not replace formal methods; it changes where formalization sits. A protocol can include temporal specifications where appropriate, and it can combine typed interfaces, executable properties, and capability manifests within a single admission artifact. PDD does not derive every implementation from first principles; it requires every admitted implementation to produce evidence that it satisfies the constraints needed for composition.

Zero Trust, Policy Systems, and Provenance

Zero-trust architecture rejects implicit trust based on location, identity, or reputation, and instead requires explicit verification (Rose et al. 2020). Policy-as-code systems such as Open Policy Agent decouple authorization logic from application code (Styra 2016). Supply-chain frameworks such as SLSA and in-toto record provenance and attestations so that artifacts can be evaluated through evidence rather than

assumption (Open Source Security Foundation (OpenSSF) 2021; Torres-Arias et al. 2019).

PDD brings that verification stance into software construction. An implementation generator proposes code, but the code must satisfy a protocol under validation. Operational invariants restrict what a module may do regardless of whether it appears functionally correct, and Evidence Chains make admission auditable rather than dependent on informal confidence.

Automated Software Engineering

The shift toward generated software was anticipated by the “Software 2.0” framing, which treats parts of software construction as search over program space rather than direct manual authorship (Karpathy 2017). Code-generating models synthesize programs from natural-language prompts (Chen et al. 2021), generative assistants can improve productivity on selected tasks (Peng et al. 2023), and repository-level agents and benchmarks make long-horizon automated software engineering measurable (Jimenez et al. 2024; Yang et al. 2024; Wu 2024).

Low-cost generation makes this problem concrete. The same prompt may produce different structures, dependencies, resource behaviors, and failure modes across models, temperatures, versions, and tool environments. PDD addresses this risk by making the prompt non-authoritative: natural language may initiate development, but the protocol determines admission.

Why a New Model Is Needed

The prior traditions reveal a consistent pattern. Specifications communicate intent but remain ambiguous; tests provide executable evidence but remain partial; formal methods provide rigor but are often selective; policy systems govern authority but usually at runtime; automated coding systems accelerate implementation but amplify variation.

This leaves a gap: a governing artifact that unifies structure, semantics, and operational authority into an admissibility boundary for generated software. PDD addresses this gap by treating implementation as a search outcome and protocol as the durable object that defines which software artifacts are admissible.

The motivating claim of PDD is not that prior models were wrong. It is that they are incomplete for automated program synthesis. When implementations are inexpensive to produce and easier to regenerate, the enduring challenge is not only how to describe or test one program, but how to define the invariants that every acceptable program must obey.

Protocol-Driven Development Model

We formalize PDD in terms of three commitments: the protocol is the primary engineering artifact, compliance gives the admission criterion, and software construction is constrained search over a protocol-defined implementation space.

The Protocol as Governing Artifact

PDD starts from the premise that implementation code is not the durable representation of a software component. The

durable representation is a machine-enforceable protocol specifying the invariant boundaries within which implementations are discovered, validated, and substituted. The protocol governs admission; implementations remain replaceable realizations.

Definition 1 (Protocol). *A protocol is a triplet*

$$\mathcal{P} = (\mathcal{S}, \mathcal{B}, \mathcal{O}),$$

where:

- $\mathcal{S}$ is the set of structural invariants;
- $\mathcal{B}$ is the set of behavioral invariants;
- $\mathcal{O}$ is the set of operational invariants.

The protocol defines the admissibility boundary for a software component. Any implementation that satisfies all three invariant classes is protocol-compliant, regardless of the algorithms, programming language, or internal organization used to realize it. The same triplet also defines the observation boundary for deployed instances: runtime evidence may evolve, but the protocol remains the governing object against which new observations are checked.

Structural Invariants

Structural invariants specify the syntactic and interface-level constraints governing communication between components.

Definition 2 (Structural Invariants). *Structural invariants $\mathcal{S}$ define rigid type and schema constraints over input and output representations, interface signatures, field types and cardinalities, and versioning rules.*

Examples include Protocol Buffers, TypeScript interfaces, JSON Schema, and OpenAPI specifications. Structural invariants ensure that all compliant implementations expose the same typed handshake.

Behavioral Invariants

Behavioral invariants specify the semantic properties that must hold across admissible executions.

Definition 3 (Behavioral Invariants). *Behavioral invariants $\mathcal{B}$ are predicates over observable behavior that must hold for all valid inputs and admissible states under the protocol's observation model.*

Examples include idempotence, determinism, monotonicity, conservation laws, and error propagation guarantees. These invariants can be encoded as property-based tests, logical assertions, metamorphic relations, or formal specifications.

Operational Invariants

Operational invariants bound the capabilities, side effects, and resource consumption of an implementation.

Definition 4 (Operational Invariants). *Operational invariants $\mathcal{O}$ specify admissible operational behavior, including constraints on external calls, dependency usage, disk and network access, execution latency, memory consumption, and concurrency.*

Examples include egress allowlists, latency and memory bounds, and dependency restrictions. They function as capability manifests or policy boundaries, restricting operational authority independently of functional correctness.

Modularity, Testability, and Operational Boundedness

The three invariant classes correspond to three governance properties. Structural invariants support *modularity* through typed handshakes; behavioral invariants support *testability* through executable or checkable predicates; operational invariants support *boundedness* by restricting authority, side effects, and resource usage. A candidate must satisfy all three to be admissible.

Implementation Space

Let $\mathcal{I}$ denote the universe of candidate implementations expressible within a given programming environment.

Definition 5 (Admissible Implementation Set). *Given protocol $\mathcal{P}$, the admissible implementation set is*

$$\mathcal{I}_{\mathcal{P}} = \{ I \in \mathcal{I} \mid I \models \mathcal{P} \}.$$

Equivalently, if $\mathcal{I}_{\mathcal{S}}$, $\mathcal{I}_{\mathcal{B}}$, and $\mathcal{I}_{\mathcal{O}}$ denote the sets of implementations satisfying the respective invariant classes, then

$$\mathcal{I}_{\mathcal{P}} = \mathcal{I}_{\mathcal{S}} \cap \mathcal{I}_{\mathcal{B}} \cap \mathcal{I}_{\mathcal{O}}.$$

Thus, a protocol defines an admissible region of implementation space, not a single implementation.

Compliance Relation

Protocol compliance is defined as conjunction across the three invariant classes.

Definition 6 (Protocol Compliance). *An implementation I is protocol-compliant if and only if*

$$I \models \mathcal{P} \iff (I \models \mathcal{S}) \wedge (I \models \mathcal{B}) \wedge (I \models \mathcal{O}).$$

The definition separates admissibility from implementation strategy: internally different implementations are equally compliant when they satisfy the same protocol-visible constraints.

Development as Constrained Search

Under Protocol-Driven Development, software construction is modeled as search over $\mathcal{I}_{\mathcal{P}}$.

An implementation generator explores candidate realizations

$$I_1, I_2, \dots, I_n \in \mathcal{I},$$

until it discovers an implementation I_k such that

$$I_k \models \mathcal{P}.$$

The implementation is accepted because it satisfies the governing constraints, not because it is manually authored, generator-preferred, or trusted by origin.

Protocol-Level Substitutability

A defining property of PDD is that implementations are replaceable with respect to the commitments made by the protocol. Compliant implementations need not be observationally identical. A protocol deliberately leaves algorithmic choices, internal data structures, or performance strategies unspecified when they are outside its commitment surface. Substitutability holds for clients whose assumptions are limited to the protocol's structural, behavioral, and operational guarantees.

Definition 7 (Protocol-Level Substitutability). *Two implementations I_a and I_b are substitutable under protocol $\mathcal{P}$ for a protocol-respecting client if both implementations satisfy the protocol,*

$$I_a \models \mathcal{P} \quad \text{and} \quad I_b \models \mathcal{P},$$

and the client depends only on guarantees entailed by $\mathcal{P}$.

Such implementations can differ internally while remaining interchangeable with respect to the guarantees the protocol actually makes. This supports regeneration, optimization, refactoring, and language migration when downstream components depend on the protocol rather than incidental behavior.

Evidence-Producing Acceptance

Compliance must be established through machine-verifiable evidence. We use *Evidence Chain* to denote the ordered record linking a protocol bundle, a candidate implementation, validator execution, Discovery Logs, signed attestations, and the resulting admission decision. For deployed implementations, the chain can be extended into a Dynamic Evidence Ledger that appends runtime attestations and violation blocks while preserving the original build-time admission evidence.

Definition 8 (Validation Function). *Let $\mathcal{E}$ be the space of valid evidence objects. Let*

$$\text{Validate}(I, \mathcal{P}) \in \mathcal{E} \cup \{\perp\}$$

be a validation function that returns an evidence object $E \in \mathcal{E}$ when the validator establishes $I \models \mathcal{P}$, and returns $\perp$ when admission fails.

The evidence object E is one signed element of the Evidence Chain, typically containing protocol versions, implementation hashes, validation outputs, and validator attestations. If validation fails, no acceptance evidence is produced. After deployment, successful and failed runtime attestations are not admission events for the current artifact; they are evidence about whether the admitted artifact continues to satisfy the monitorable projection of $\mathcal{P}$ under live observations.

Summary

In PDD, software construction centers on the protocol, the object that combines structural, behavioral, and operational invariants into an admissible implementation space. The protocol, rather than any specific implementation, becomes the authoritative admission contract for a component. Implementations are accepted only when accompanied by machine-verifiable evidence of compliance.

Protocol Authoring and Disambiguation

Protocol-Driven Development begins by translating architectural intent into machine-enforceable constraints. Informal requirements, design discussions, or prompts can initiate the process, but the authoritative artifact is the protocol: a specification whose admissibility conditions are machine-validatable.

Protocol authoring employs three mechanisms: *typed handshakes* for structure, *property-based assertions* for behavior,

and *capability manifests* for operational authority. Together they map prose to restrictive assertions over structure, semantics, and side effects.

The Natural Language Tax

The *Natural Language Tax* is the recurring cost imposed when durable engineering intent is represented primarily in prose. It appears as *interpretation cost*, when developers or agents infer types, edge cases, capability boundaries, and failures from underspecified statements; *reconciliation cost*, when incompatible interpretations are repaired after implementation begins; and *maintenance cost*, when prose drifts as implementations evolve.

PDD reduces this tax by making prose a staging artifact rather than the final control surface. Natural language may initiate protocol authoring, but the durable output is a set of machine-enforceable constraints: typed handshakes for representation, property-based assertions for behavior, and capability manifests for side effects and authority.

Typed Handshakes

A PDD protocol begins with a *typed handshake*: a machine-readable contract specifying inputs, outputs, errors, and compatibility rules. Typed handshakes can be written in JSON Schema, OpenAPI, Protocol Buffers, TypeScript interfaces, or equivalent schema languages (JSON Schema 2022; OpenAPI Initiative 2021; Google 2008).

A typed handshake reduces *type drift* between architectural intent, generated implementation, and downstream consumers. Instead of saying “returns a user object,” PDD defines a versioned schema with required and optional fields, enumerations, nullability, error variants, and deprecation rules. Ambiguity becomes a compile-time or validation failure.

As long as the handshake is preserved, protocol-compliant implementations remain interchangeable at the structural boundary.

Property-Based Assertions

Protocol authoring next elevates example-based expectations into general behavioral laws. Instead of stating informally that “the handler should be idempotent,” the protocol encodes the invariant directly as

$$\forall x \in X, \quad f(f(x)) = f(x).$$

Similarly, rather than saying that “invalid input should fail safely,” the protocol defines

$$\text{invalid}(x) \Rightarrow \text{isError}(f(x)).$$

These properties become validator targets for property-based testing frameworks, symbolic analysis tools, theorem provers, or runtime validation mechanisms. Regardless of the enforcement mechanism, the invariant itself becomes the authoritative statement of intended behavior.

Property-based assertions reduce *intent drift*: they prevent an implementation from satisfying visible examples while violating the general property those examples were meant to express. In PDD, the acceptance criterion is not a curated test list, but the invariant itself.

Capability Manifests

The third step specifies what the implementation is allowed to do. A *capability manifest* defines operational authority and resource boundaries, including:

- file-system access;
- outbound network destinations;
- database operations and transaction limits;
- maximum external calls per request;
- memory and CPU budgets;
- latency targets;
- environment variables and secret access;
- concurrency bounds;
- permitted background work.

These constraints become enforcement targets for sandboxing, runtime instrumentation, policy engines, operating system controls, or deployment configuration.

Capability manifests reduce *side-effect drift*. An implementation may pass functional tests while introducing hidden caches, third-party API calls, temporary file writes, or expanded transaction scopes. Under PDD, such behavior is part of the protocol: if a module has not been granted disk I/O, any write attempt is a violation regardless of functional output.

Evidence Chains

The final mechanism bridges the gap between protocol definition and verifiable acceptance. An *Evidence Chain* is a signed record that binds a specific implementation artifact to the protocol it satisfies, the validator that assessed it, and the Discovery Log of its operational characteristics.

Evidence Chains reduce *audit drift*: the divergence between running software and the rationale for why it was admitted. Instead of relying on statements such as “it passed all tests locally,” the Evidence Chain preserves a machine-checkable record of compliance.

Constraining Ambiguity

Protocol-Driven Development replaces descriptive statements with restrictive assertions. Human authors can still begin with prose, but prose is not authoritative. Each common class of ambiguity is mapped to a corresponding machine-enforceable mechanism, as summarized in Table 1.

The mapping moves ambiguity from social interpretation into enforceable structure. The architect need not prescribe every implementation detail; the architect defines the admissibility conditions that all valid implementations must satisfy.

From Narrative to Law

PDD moves from descriptive intent to formal constraint. Once structural, behavioral, and operational invariants are encoded as machine-enforceable constraints, implementations can be generated, discarded, and regenerated without changing the authoritative artifact. Protocol authoring therefore becomes the main design activity: the architect defines the invariant boundaries within which acceptable implementations are discovered.

Validator Loop and Evidence Chains

The *Validator Loop* is the admission cycle for PDD. It separates protocol definition, candidate generation, validation, and admission. The generator searches; the protocol and validator define success. In deployed systems, the same evidence discipline continues through runtime attestation, but runtime observations do not bypass admission: repaired or regenerated artifacts still return to the Validator Loop before replacement.

Protocol Authors define admissibility conditions, Implementation Generators propose candidates, Validation Engines evaluate candidates, and Evidence Stores preserve the basis for acceptance. A software artifact becomes admissible only after compliance is evaluated and recorded.

Contract Negotiation

The Validator Loop begins when a *Protocol Author* proposes a protocol $\mathcal{P}$. Because modules rarely exist in isolation, dependencies must be reconciled into a coherent admissibility boundary before implementation begins.

Contract negotiation verifies that the typed handshakes, behavioral properties, and operational capabilities of dependent protocols are mutually compatible. It includes dependency resolution, compatibility checking, capability reconciliation, and conflict detection across transitive protocol boundaries.

The output of negotiation is a *versioned protocol bundle* containing:

- structural schemas and interface definitions;
- behavioral properties and regression constraints;
- operational capability manifests;
- protocol dependencies and compatibility metadata;
- approved validator implementations and versions.

Once sealed, the bundle becomes the target for generation. Generators are not permitted to silently weaken or reinterpret its constraints; protocol changes require an explicit version event and renewed negotiation.

The illustrative bundle in Appendix shows one compact representation of these artifacts without prescribing a particular format.

Automated Generation

An *Implementation Generator* receives the protocol bundle and searches for a candidate implementation using prompting, retrieval, synthesis, repair, evolutionary search, templates, or other strategies. PDD is agnostic to model, language, and toolchain.

A candidate implementation I is treated as untrusted until it has passed validation against the protocol bundle.

Formally, the Implementation Generator explores a sequence

$$I_1, I_2, \dots, I_n \in \mathcal{I},$$

until it discovers some implementation I_k such that

$$I_k \models \mathcal{P}.$$

The loop permits multiple generators to compete against the same protocol and regenerated implementations to replace earlier ones without changing dependent contracts. The

Table 1: Protocol-Driven Development mapping of ambiguity classes.

Mechanism	Illustrative Narrative Requirement	Drift Removed
Typed handshake	“Returns a user object”	Type drift
Property-based assertion	“Handles invalid input safely”	Intent drift
Capability manifest	“Do not call the database too much”	Side-effect drift
Evidence Chain	“It passed on my machine”	Audit drift

implementation is a provisional witness to satisfiability, not the durable artifact.

Verification

A *Validation Engine* performs verification: it inspects artifacts, executes validation logic, monitors resources, and rejects non-compliant candidates. It acts as the admission controller for the implementation space.

Verification proceeds in three layers:

1. **Structural validation** checks that interfaces compile, serialize, and conform to $\mathcal{S}$.
2. **Behavioral validation** checks properties, examples, metamorphic relations, and regression suites against $\mathcal{B}$.
3. **Operational validation** executes the candidate under policy, sandboxing, or instrumentation to verify compliance with $\mathcal{O}$.

The layers are jointly necessary: structural checks alone miss semantics, behavioral checks leave hidden capability violations unobserved, and operational checks cannot establish functional meaning.

Because validators define the acceptance boundary, they must be trusted at least as much as the build and release system. In high-assurance settings, validators may themselves be versioned, sandboxed, reproducible, and attested.

Discovery Logs

When validation succeeds, the admitted implementation emits a *Discovery Log*: an as-built record of what was produced and observed. The protocol states what *must* be true; the Discovery Log records what *was found* to be true of this implementation.

A Discovery Log includes entries such as:

- implementation language and compiler versions;
- dependency graph and package hashes;
- generated files and artifact digests;
- validator identities and versions;
- property coverage and validation outcomes;
- observed resource usage;
- derived behaviors not explicitly enumerated in the original protocol.

Discovery Logs make acceptance auditable and provide feedback for protocol evolution. Useful recurring behaviors can be promoted into future protocol versions; undesirable behaviors can motivate stronger constraints.

Evidence Chains

An *Evidence Chain* is the ordered record linking protocol constraints, generated implementations, validation outcomes, and deployment artifacts. Within PDD, it binds software admission to accountable evidence.

Let the validator produce an evidence object

$$E = H(\mathcal{P}, I, V, R, t),$$

where:

- $\mathcal{P}$ is the negotiated protocol bundle;
- I is the admitted implementation artifact;
- V identifies the validator implementations and versions;
- R contains validation results and measured observations;
- t records time, environment, and provenance metadata;
- H denotes a cryptographic digest or signed attestation over these elements.

The evidence object supports audit, end-to-end accountability, and replay whenever the relevant validator inputs are preserved. A downstream system asks whether the artifact is linked to an approved protocol and whether the Evidence Chain records satisfaction under approved validators.

The same evidence structure links development and runtime governance when deployment systems consume development-time evidence before runtime admission. Once an implementation is deployed, runtime monitors can append additional evidence blocks to the same logical record. These blocks may attest continued compliance, record bounded degradation, or capture invariant violations for later repair. Section formalizes this extension as a Dynamic Evidence Ledger.

Acceptance as an Evidence-Producing Event

In PDD, validation is the mechanism by which software becomes admissible. An implementation is accepted if and only if:

1. a protocol bundle has been negotiated and sealed;
2. a candidate implementation has been generated;
3. the candidate satisfies all structural, behavioral, and operational invariants; and
4. the validator emits signed evidence linking the artifact to the governing protocol.

The admission principle is:

Code is accepted not because it appears correct, but because compliance has been validated and recorded.

Under this model, each admitted implementation carries a verifiable chain showing why it was permitted to enter the system.

Summary

The Validator Loop separates proposal, verification, and admission. Protocol bundles define admissibility, generators explore candidates, validators establish compliance, Discovery Logs record observations, and Evidence Chains preserve the basis for admission.

Theoretical Foundations

The formal account below focuses on the core PDD relations. It does not prove that every useful software property is decidable or that validation is complete. Rather, it states what follows when protocols define the relevant observation boundary and validators are sound with respect to that boundary.

Implementation-Space Interpretation

Let $\mathcal{I}$ denote the set of candidate implementations expressible by a generator within a target environment. The set includes programs produced through prompting, retrieval, synthesis, repair, mutation, or any other search strategy. A natural-language prompt defines at most an imprecise region of $\mathcal{I}$: different models, sampling temperatures, prompts, or tool contexts may produce implementations that appear plausible while diverging in structure, behavior, dependencies, and operational footprint.

PDD replaces this open-ended search with an explicit admissibility boundary. For a protocol

$$\mathcal{P} = (\mathcal{S}, \mathcal{B}, \mathcal{O}),$$

let $\mathcal{I}_{\mathcal{S}}$, $\mathcal{I}_{\mathcal{B}}$, and $\mathcal{I}_{\mathcal{O}}$ denote the implementations satisfying the structural, behavioral, and operational invariant classes, respectively. The admissible implementation set induced by $\mathcal{P}$ is

$$\mathcal{I}_{\mathcal{P}} = \mathcal{I}_{\mathcal{S}} \cap \mathcal{I}_{\mathcal{B}} \cap \mathcal{I}_{\mathcal{O}}.$$

Thus, PDD models software construction as constrained search from plausible implementations to admissible implementations. The protocol does not define a single program; it defines the region of implementation space within which candidate programs are admitted.

Observation-Model Semantics

The satisfaction relation $I \models \mathcal{P}$ is observational rather than intensional. Compliance is not defined by the private internal organization of I , but by the observations that $\mathcal{P}$ makes relevant.

Definition 9 (Protocol Observation Model). *For protocol $\mathcal{P}$, let $\Omega_{\mathcal{P}}$ be the set of protocol-relevant observations and let $\Phi_{\mathcal{P}}$ be the predicate that characterizes admissible observations. For implementation I , let $\text{Obs}_{\mathcal{P}}(I) \subseteq \Omega_{\mathcal{P}}$ denote the observations of I visible under the protocol’s observation model.*

The observation set includes the protocol-relevant structural, behavioral, and operational observations: schema conformance and serialization behavior; outputs, errors, invariants, and temporal relations; and resource use, side effects,

external calls, and capability boundaries. The protocol predicate decomposes as

$$\Phi_{\mathcal{P}}(\omega) \iff \Phi_{\mathcal{S}}(\omega) \wedge \Phi_{\mathcal{B}}(\omega) \wedge \Phi_{\mathcal{O}}(\omega),$$

where each component corresponds to one invariant class.

Definition 10 (Protocol Satisfaction). *An implementation I satisfies protocol $\mathcal{P}$, written $I \models \mathcal{P}$, if and only if*

$$\forall \omega \in \text{Obs}_{\mathcal{P}}(I), \quad \Phi_{\mathcal{P}}(\omega).$$

This definition permits heterogeneous implementations. A Python service, a Rust binary, and a generated WebAssembly module can all satisfy the same protocol when their protocol-visible observations satisfy the same structural, behavioral, and operational predicates. Outside the protocol’s commitment surface, they need not be identical.

Validators and Evidence-Carrying Admission

A validator is the mechanism that decides whether a candidate enters $\mathcal{I}_{\mathcal{P}}$. We write

$$\text{Validate}(I, \mathcal{P}) \in \mathcal{E} \cup \{\perp\},$$

where $\mathcal{E}$ is the space of valid evidence objects and $\perp$ denotes rejection.

Definition 11 (Validator Soundness). *A validator for protocol $\mathcal{P}$ is sound if, whenever $\text{Validate}(I, \mathcal{P}) = E$ with $E \neq \perp$, it follows that $I \models \mathcal{P}$.*

Theorem 1 (Evidence-Carrying Sound Admission). *Assume a sound validator for $\mathcal{P}$ and an evidence object $E \in \mathcal{E}$ that cryptographically binds the protocol identity, implementation identity, validator identity, and validation result. If $\text{Validate}(I, \mathcal{P}) = E$ and $E \neq \perp$, then $I \in \mathcal{I}_{\mathcal{P}}$. Moreover, the evidence cannot be interpreted as an admission of a different implementation or a different protocol without breaking the binding assumption.*

Proof. Since the validator is sound and returns $E \neq \perp$, we have $I \models \mathcal{P}$. By the definition of the admissible implementation set, $I \in \mathcal{I}_{\mathcal{P}}$. The second claim follows from the assumed binding of E to the protocol identity, implementation identity, validator identity, and validation result. $\square$

The theorem is intentionally conditional. If the validator is unsound, if the protocol omits a relevant property, or if the evidence binding is compromised, PDD does not provide the stated admission guarantee. The formal claim is that evidence-bearing admission is meaningful only relative to a specified protocol and a sound validation boundary.

Protocol-Respecting Clients

Substitutability in PDD is not full observational equivalence; it is equivalence relative to the guarantees exposed by the protocol. Let $G(\mathcal{P})$ denote the set of guarantees entailed by $\mathcal{P}$ under its observation model.

Definition 12 (Protocol-Respecting Client). *A client C is protocol-respecting with respect to $\mathcal{P}$ if every assumption C makes about a component is contained in $G(\mathcal{P})$. Such a client is permitted to rely on the protocol’s structural, behavioral, and operational guarantees, but not on implementation internals or behavior outside the protocol’s commitment surface.*

Theorem 2 (Protocol-Level Substitutability). *Let $I_a, I_b \in \mathcal{I}_{\mathcal{P}}$. For any client C that is protocol-respecting with respect to $\mathcal{P}$, replacing I_a with I_b preserves every client obligation that depends only on $G(\mathcal{P})$.*

Proof. Since $I_a, I_b \in \mathcal{I}_{\mathcal{P}}$, both satisfy $\mathcal{P}$ and therefore satisfy the guarantees in $G(\mathcal{P})$. Because C is protocol-respecting, its component-facing assumptions are limited to $G(\mathcal{P})$. Replacing I_a with I_b therefore preserves all assumptions on which C is permitted to depend. $\square$

This result does not claim that I_a and I_b are identical in latency, internal state layout, dependency choices, or behavior that the protocol leaves unspecified. Such differences are admissible precisely when they lie outside the protocol’s commitment surface.

Theorem 3 (Safe Regeneration Under Protocol-Respecting Dependency). *Let I_{old} be an implementation admitted for protocol $\mathcal{P}$ by a sound validator, and let I_{new} be a regenerated implementation such that $I_{\text{new}} \in \mathcal{I}_{\mathcal{P}}$. If all downstream dependencies are protocol-respecting with respect to $\mathcal{P}$, then replacing I_{old} with I_{new} preserves all downstream obligations expressible in $G(\mathcal{P})$.*

Proof. Because I_{old} is admitted under $\mathcal{P}$ by a sound validator, evidence-carrying sound admission gives $I_{\text{old}} \in \mathcal{I}_{\mathcal{P}}$. By assumption, $I_{\text{new}} \in \mathcal{I}_{\mathcal{P}}$. Applying protocol-level substitutability to each protocol-respecting downstream dependency shows that replacement preserves every dependency obligation whose assumptions are contained in $G(\mathcal{P})$. $\square$

Safe regeneration is therefore not a claim about arbitrary clients. It holds only for dependencies whose assumptions are constrained to the protocol. If a downstream component depends on accidental behavior of the old implementation, regeneration may expose that invalid coupling.

Protocol Refinement

Protocol refinement evolves protocols by adding constraints. We write $\mathcal{P}' \succeq \mathcal{P}$ when $\mathcal{P}'$ preserves every constraint of $\mathcal{P}$ and adds further structural, behavioral, or operational constraints. Equivalently, every observation admitted by $\mathcal{P}'$ is also admitted by $\mathcal{P}$.

Proposition 1 (Refinement Narrows Admissibility). *If $\mathcal{P}' \succeq \mathcal{P}$, then*

$$\mathcal{I}_{\mathcal{P}'} \subseteq \mathcal{I}_{\mathcal{P}}.$$

Proof. For any implementation I , if $I \models \mathcal{P}'$, then all observations of I satisfy the stronger predicate $\Phi_{\mathcal{P}'}$. Since $\mathcal{P}'$ preserves every constraint of $\mathcal{P}$, those observations also satisfy $\Phi_{\mathcal{P}}$, so $I \models \mathcal{P}$. Hence every implementation in $\mathcal{I}_{\mathcal{P}'}$ is also in $\mathcal{I}_{\mathcal{P}}$. $\square$

Refinement captures the governance effect of protocol evolution. Adding constraints can exclude previously admissible implementations, but it cannot enlarge the admissible set unless the protocol is weakened rather than refined.

Summary

The formal role of PDD is to separate implementation search from artifact admission. Protocols define an observation boundary, validators provide conditional evidence that a candidate lies within that boundary, protocol-respecting clients can substitute or regenerate implementations without depending on incidental behavior, and refinement monotonically narrows the admissible implementation set. These results make precise the central claim of the paper: code is transient, while the protocol is the durable representation of software governance.

Reference Architecture

The reference architecture identifies minimal components for separating design intent, implementation search, validation, runtime enforcement, and evidence preservation without prescribing an implementation stack.

Architectural Overview

A minimal PDD system consists of seven principal components:

1. **Protocol Author**, which authors and evolves protocols;
2. **Protocol Registry**, which stores versioned protocol bundles;
3. **Implementation Generator**, which searches for candidate realizations;
4. **Validation Engine**, which verifies protocol compliance;
5. **Runtime Verification Layer**, which observes and enforces protocol invariants during live execution;
6. **Evidence Store**, which preserves Discovery Logs, Evidence Chain artifacts, and Dynamic Evidence Ledger blocks;
7. **Remediation Orchestrator**, which converts runtime violations into repair contexts for renewed generation and validation.

The components form a closed governance loop: protocols define admissibility, generators search, validators decide admission, runtime verifiers monitor deployed behavior, evidence stores record the basis of each judgment, and remediation returns failures to the generation pipeline.

The artifact sketch in Appendix connects these roles to concrete protocol files, validator declarations, and evidence records without treating the sketch as a deployed prototype.

Protocol Author

The *Protocol Author* translates intent into structural, behavioral, and operational invariants; negotiates dependencies; resolves compatibility conflicts; and versions protocol bundles. The role can be human, automated, or hybrid. Instead of prescribing one implementation, it defines the invariant boundary within which implementations are discovered and substituted.

Protocol Registry

The *Protocol Registry* stores versioned protocol bundles: typed handshakes, behavioral invariants, capability manifests, dependency declarations, validator requirements, and provenance metadata. It is the durable system of record for protocol evolution.

Implementation Generator

The *Implementation Generator* produces candidate realizations of a protocol bundle using generative models, synthesis, templates, or other search strategies. It operates outside the admission boundary: its outputs are proposals, and it has no authority to declare them valid.

Validation Engine

The *Validation Engine* evaluates candidate artifacts against the governing protocol for structural, behavioral, and operational compliance. As required by the protocol, it invokes compilers, property-based testing, symbolic analyzers, sandbox monitors, policy engines, or provenance tooling. Generation proposes; validation decides.

Runtime Verification Layer

The *Runtime Verification Layer* observes deployed executions from outside the generated implementation. It checks live payloads, state transitions, dependency calls, and resource measurements against the monitorable subset of the same structural, behavioral, and operational invariant classes used for admission. When an execution violates the protocol, the layer can block, quarantine, rate-limit, roll back, or emit a violation block for the Dynamic Evidence Ledger.

Evidence Store

The *Evidence Store* preserves artifacts required to audit acceptance decisions and replay them when validator inputs are available: Discovery Logs, signed evidence objects, validator traces, implementation hashes, provenance records, runtime attestation blocks, and violation reports.

Remediation Orchestrator

The *Remediation Orchestrator* turns runtime violation evidence into a structured repair context. It identifies the violated protocol clause, gathers relevant telemetry and environment metadata, and returns the failure to the Implementation Generator. The resulting patch or regenerated artifact is treated as a new candidate and must pass the Validation Engine before deployment.

End-to-End Flow

The end-to-end flow proceeds as follows:

1. The Protocol Author defines and seals a protocol bundle.
2. The Protocol Registry publishes the bundle as the authoritative component contract.
3. The Implementation Generator retrieves the bundle and generates candidate realizations.
4. The Validation Engine evaluates each candidate against the protocol.

5. Upon successful validation, the system emits a Discovery Log and a signed evidence object.
6. The Evidence Store records the resulting artifacts as part of the Evidence Chain.
7. Approved implementations are released, deployed, or made available for substitution.
8. The Runtime Verification Layer monitors deployed behavior and appends attestation blocks to the Dynamic Evidence Ledger.
9. Runtime violations are converted into repair contexts, and any proposed fix re-enters validation before replacement.

This treats software construction and operation as a continuous governance workflow.

Trust Boundaries and Interoperability

The reference architecture defines explicit admission and enforcement boundaries: generators and code artifacts are untrusted by default, protocols, validators, runtime verifiers, and evidence stores form the control plane, and deployment systems are conditionally trusted to verify evidence before runtime admission.

The architecture augments existing toolchains. Typed handshakes map to OpenAPI and Protocol Buffers, behavioral invariants integrate with property-based testing and formal verification, operational invariants use policy engines, runtime verifiers can be implemented with sidecars, gateways, service meshes, sandbox monitors, or kernel instrumentation, and Evidence Chains align with supply-chain frameworks such as SLSA.

Summary

The reference architecture decomposes software construction and operation into explicit roles: authors define admissibility, registries preserve protocols, generators search, validators establish compliance, runtime verifiers enforce deployed invariants, remediation components return failures to generation, and evidence stores preserve the basis for each admission and attestation decision.

Runtime Evidence and Continuous Attestation

The Validator Loop establishes that a generated artifact is admissible at the time it enters the system. That claim is necessary but not sufficient for production governance. A deployed implementation may later encounter inputs, traffic patterns, dependency states, or concurrency schedules absent from pre-deployment validation. PDD therefore extends admission evidence into runtime evidence: the protocol remains the governing artifact, but evidence continues to accumulate after deployment.

Runtime evidence is necessarily scoped by observability. Some protocol obligations can be checked online from payloads, traces, resource counters, policy decisions, or state-transition events. Other obligations may require offline replay, sampling, formal analysis, or stronger instrumentation. Continuous attestation therefore covers the monitorable projection of the protocol rather than replacing build-time validation.

The Static Evidence Boundary

Build-time validation creates a controlled observation boundary. Validators can compile artifacts, execute property suites, inspect dependencies, run sandboxed workloads, and measure resource use under known conditions. These checks support disciplined admission, but they cannot exhaust the space of production executions. Live systems exhibit workload drift, partial failures, adversarial inputs, scheduler interleavings, dependency changes, and gradual resource degradation.

The limitation is not a defect in the Evidence Chain; it is a boundary condition. A development-time evidence object proves compliance relative to preserved validation inputs, validator assumptions, and the protocol observation model. It should not be read as a perpetual guarantee that every future execution will remain compliant.

Dynamic Evidence Ledgers

To represent compliance across the operational lifetime of a component, PDD generalizes the Evidence Chain into a *Dynamic Evidence Ledger*. The initial ledger element records build-time admission:

$$\mathcal{L}_0 = (E_{\text{build}}).$$

At runtime, each attestation interval produces a signed evidence block:

$$E_t = H(E_{t-1}, \mathcal{P}, I_v, R_t, A_t, t),$$

where E_{t-1} is the previous evidence block, $\mathcal{P}$ is the governing protocol, I_v identifies the deployed implementation version, R_t contains runtime observations over interval t , A_t records the attestation decision, and H denotes a cryptographic digest or signed attestation. The ledger evolves append-only:

$$\mathcal{L}_t = \mathcal{L}_{t-1} \parallel E_t.$$

The protocol itself remains the triplet

$$\mathcal{P} = (\mathcal{S}, \mathcal{B}, \mathcal{O}).$$

The deployed instance is time-indexed by implementation and evidence state:

$$\mathcal{D}_t = (\mathcal{P}, I_v, \mathcal{L}_t).$$

This distinction preserves the protocol as the durable specification while allowing operational evidence to evolve.

Definition 13 (Monitorable Runtime Projection). *For protocol $\mathcal{P}$, let $\Omega_{\mathcal{P}}^r \subseteq \Omega_{\mathcal{P}}$ denote the protocol-relevant observations available to the runtime verifier. The runtime predicate $\Phi_{\mathcal{P}}^r$ is the restriction of $\Phi_{\mathcal{P}}$ to $\Omega_{\mathcal{P}}^r$.*

Definition 14 (Continuous Protocol Attestation). *For protocol $\mathcal{P}$ and deployed implementation version I_v , continuous attestation over interval t succeeds when every runtime observation in $R_t \subseteq \Omega_{\mathcal{P}}^r$ satisfies $\Phi_{\mathcal{P}}^r$, and the resulting decision is appended to $\mathcal{L}_t$ as signed evidence.*

Continuous attestation therefore changes the interpretation of compliance from a one-time admission event to a sequence of recorded claims over monitorable behavior. A component is not merely accepted once; it remains accountable to the protocol while it executes.

Runtime Verification Layer

Runtime evidence requires an enforcement boundary outside the generated implementation. We call this boundary the *Runtime Verification Layer* (RVL). It can be realized through sidecars, service mesh policies, gateways, admission controllers, eBPF instrumentation, sandbox monitors, or other deployment mechanisms. The key property is isolation: the generated implementation may produce behavior, but it cannot unilaterally decide whether that behavior is compliant.

The RVL performs three functions:

1. **Runtime observation:** collect protocol-relevant payloads, state transitions, resource measurements, dependency calls, and policy decisions.
2. **Runtime enforcement:** block, quarantine, rate-limit, roll back, or degrade executions that violate structural, behavioral, or operational invariants.
3. **Runtime attestation:** append signed evidence blocks to the Dynamic Evidence Ledger, including both successful attestations and violations.

This layer is aligned with runtime verification: executions are monitored against formal or executable properties while the system runs (Leucker and Schallhart 2009; Havelund and Roşu 2004). PDD uses that pattern as part of a generated-software governance loop rather than as an isolated monitoring technique.

Definition 15 (Runtime Verifier Soundness). *A runtime verifier for $\mathcal{P}$ is sound with respect to $\Phi_{\mathcal{P}}^r$ if every emitted pass block corresponds to observations satisfying $\Phi_{\mathcal{P}}^r$, and every emitted violation block identifies at least one observed $\omega \in \Omega_{\mathcal{P}}^r$ such that $\neg\Phi_{\mathcal{P}}^r(\omega)$.*

Proposition 2 (Evidence-Carrying Runtime Violation). *Assume a sound runtime verifier for $\mathcal{P}$ and a violation block E_t^{fail} that cryptographically binds the deployed implementation version, runtime observation, protocol identity, verifier identity, and violated predicate. Then E_t^{fail} is evidence of non-compliance with the monitorable runtime projection of $\mathcal{P}$ for the recorded interval.*

Proof. By runtime verifier soundness, an emitted violation block identifies an observed $\omega \in \Omega_{\mathcal{P}}^r$ for which $\neg\Phi_{\mathcal{P}}^r(\omega)$. The cryptographic binding prevents the block from being reassigned to a different implementation, protocol, verifier, or observation without breaking the binding assumption. The block therefore records non-compliance with the monitorable projection of $\mathcal{P}$ for that interval. $\square$

Runtime Anomaly Taxonomy

Runtime attestation is motivated by anomaly classes that are difficult to eliminate through pre-deployment validation alone:

- **Structural drift:** live payloads, schema versions, or serialization behavior diverge from the typed handshake.
- **Behavioral drift:** observed outputs, error semantics, ordering guarantees, or monitorable idempotence properties diverge from $\mathcal{B}$.

- **Operational degradation:** latency, memory use, retries, cost, or dependency fan-out exceed $\mathcal{O}$ under production load.
- **Temporal and concurrency anomalies:** races, ordering violations, deadlocks, livelocks, or stale reads appear only under particular schedules.
- **Dependency and environment drift:** external services, package versions, configuration, or infrastructure state change after admission.
- **Authority and provenance violations:** an implementation attempts unapproved network access, file writes, privilege escalation, or unrecorded dependency use.
- **Distribution and adversarial shift:** production inputs exploit assumptions not represented in validation workloads.

These classes do not replace protocol authoring; they identify where protocols and validators may need refinement. A recurring anomaly can become a new structural, behavioral, or operational invariant in a later protocol version.

Closed-Loop Remediation

When runtime attestation fails, the RVL should not merely emit an alert. It should produce a structured invariant failure context that can be used by the generation and validation pipeline. Evidence blocks may store redacted observations, hashes, or pointers to access-controlled traces when raw telemetry contains sensitive data. Let

$$E_t^{\text{fail}} = H(E_{t-1}, \mathcal{P}, I_v, R_t^{\text{fail}}, A_t^{\text{fail}}, t)$$

denote a signed violation block. The remediation loop is:

$$E_t^{\text{fail}} \rightarrow C_t \rightarrow I' \rightarrow \text{Validate}(I', \mathcal{P}) \rightarrow \mathcal{L}_{t+1},$$

where C_t is a repair context derived from the violating observation, protocol clause, environment metadata, and ledger state. A generator may use C_t to propose a patch I' , but the patch is not trusted because it was generated in response to a real failure. It must re-enter the Validator Loop and produce fresh admission evidence before deployment.

This closes the self-correction loop. Runtime failure becomes protocol-indexed evidence, not an informal bug report. The generator receives concrete proof of non-compliance, while the validator and RVL retain authority over admission and enforcement.

Summary

Dynamic Evidence Ledgers, Runtime Verification Layers, and closed-loop remediation extend PDD from static admission to continuous protocol governance. The protocol remains sovereign, the generator remains a proposal engine, and runtime execution becomes another source of signed evidence about whether observed deployed behavior remains within the monitorable protocol boundary.

Case Studies

We illustrate PDD with three examples: an idempotent handler, a bounded ETL pipeline, and an automatically generated microservice. These demonstrate how protocols govern implementations across scales.

Idempotent User-Creation Handler

Consider a service that creates a user account from a client-supplied identifier. In a conventional design document, the requirement might be written informally as:

“Create the user if it does not already exist, and return the existing record otherwise.”

This statement leaves open required fields, duplicate recognition, admissible errors, external lookups, retries, and operational budgets.

Under PDD, these requirements are expressed as a protocol:

- **Structural invariants ($\mathcal{S}$):** The request and response are defined by explicit schemas specifying required fields, optional metadata, and enumerated error variants.
- **Behavioral invariants ($\mathcal{B}$):** Repeated invocations with the same logical identifier must be idempotent:

$$f(x, s) = (y, s') \Rightarrow f(x, s') = (y, s').$$

Additional properties include deterministic errors and identifier uniqueness.

- **Operational invariants ($\mathcal{O}$):** At most one database write is permitted; no outbound network access is allowed; and end-to-end latency must remain below a specified bound.

An implementation generator is free to use optimistic insertion, uniqueness constraints, transactional lookup, or an equivalent strategy. The validation engine checks schema conformance, idempotence, error behavior, and operational limits; once admitted, the implementation can later be replaced without changing the protocol.

Bounded ETL Pipeline

As a second example, consider an extract-transform-load (ETL) component that normalizes transactional records before downstream analysis.

In a conventional specification, the requirement might be: “process all valid records and reject malformed inputs.” This leaves ordering, determinism, temporary disk usage, and memory budgets unspecified.

Under PDD, the pipeline is instead governed by a protocol that makes these constraints explicit:

- **Structural invariants:** Input and output schemas define exact field types, nullability rules, and admissible record formats.
- **Behavioral invariants:** The transformation must be deterministic, schema-preserving where required, and conservative with respect to valid-record counts unless filtering or aggregation is explicitly permitted.
- **Operational invariants:** The pipeline must execute in streaming mode, remain within a fixed memory budget, avoid temporary disk writes, and satisfy a maximum per-record processing latency.

The implementation generator is free to realize the pipeline in Python, Rust, Apache Beam, or another framework. These choices are secondary; the generated implementation must satisfy the protocol’s structural, behavioral, and operational constraints.

Automated Microservice Generation

The third case study illustrates the full Protocol-Driven Development lifecycle in an automated synthesis setting.

Suppose an architect introduces a fraud-detection microservice by defining a protocol bundle containing:

- gRPC request and response schemas;
- behavioral properties such as deterministic scoring, monotonicity of risk under added evidence, and well-defined failure semantics;
- operational constraints such as latency budgets, approved feature stores, restricted outbound network access, and dependency on existing authentication and audit protocols.

An implementation generator retrieves the bundle and produces candidates using different prompts, libraries, or internal architectures. Each candidate is treated as a provisional proposal.

The validation engine checks interface conformance, determinism, monotonicity, regression properties, approved feature access, latency, and dependency constraints. Non-compliant candidates are rejected.

If a candidate satisfies the protocol bundle, it is admitted and recorded with its Discovery Log and signed evidence object. After deployment, a Runtime Verification Layer monitors feature-store calls, latency, response shape, and policy decisions. Suppose a production traffic shift causes the service to issue two feature-store calls for a subset of requests, violating the operational invariant that allows only one such call. The RVL records the violating trace as a signed ledger block and can rate-limit or quarantine the offending path. The remediation orchestrator converts the block into repair context for the generator; any proposed fix must pass the original Validator Loop before replacement.

Later, a new generator can also produce a more efficient realization; if the protocol is unchanged and the new artifact validates, the new version can replace the original without weakening the stated guarantees.

Comparative Lessons

Four properties recur: the protocol defines the stable component surface; regeneration and substitution become explicit operations; operational boundaries become first-class elements; and the same admission logic applies from functions to services.

Implications

The examples demonstrate incremental application of PDD, beginning with small module boundaries and extending to service architectures. PDD stabilizes what must remain stable: interface boundaries, behavioral properties, operational authority, and evidence of compliance.

Evaluation Agenda

PDD requires an empirical agenda focused on admission quality rather than code-generation speed alone. Implementations should be judged by whether protocol-governed construction yields artifacts that are less ambiguous, easier to regenerate, more substitutable, more operationally bounded, and more auditable than artifacts governed by prose and tests alone.

Empirical Questions

Future evaluations should organize around six questions:

1. **Ambiguity Reduction:** Do protocols reduce implementation variance relative to narrative specifications and test suites alone?
2. **Regeneration Reliability:** Can independently generated implementations repeatedly satisfy the same protocol across models, prompts, and languages?
3. **Protocol-Level Substitutability:** Are distinct protocol-compliant implementations interchangeable for protocol-respecting clients?
4. **Validation Overhead:** What computational and engineering costs are introduced by the Validator Loop?
5. **Governance Efficacy:** Does protocol-based admission detect and block unauthorized side effects, capability violations, and provenance gaps?
6. **Runtime Attestation:** Do runtime verifiers and Dynamic Evidence Ledgers detect post-deployment anomalies and produce repair evidence?

These claims are empirically falsifiable. If protocols do not reduce protocol-visible variance, admitted implementations cannot be regenerated reliably, compliant implementations are not substitutable for protocol-respecting clients, the Validator Loop misses routine capability violations, or runtime attestation fails to detect monitorable post-deployment anomalies, then the central claims of PDD would require revision.

Workloads and Baselines

An empirical validation program should span multiple levels of system complexity:

- **Functions:** idempotent handlers, parsers, validators, and deterministic transformations;
- **Data pipelines:** bounded ETL jobs with explicit memory, latency, and side-effect constraints;
- **Microservices:** gRPC and REST services with persistence, dependency policies, and runtime authority restrictions;
- **Automated regeneration tasks:** repeated implementation of the same protocol across multiple models, prompting strategies, and programming languages.

For each workload, future studies should compare at least three conditions:

1. **Spec-Driven Development (SDD):** natural-language requirements and interface descriptions;
2. **Test-Driven Development (TDD):** requirements supplemented by executable tests;
3. **Protocol-Driven Development (PDD):** structural, behavioral, and operational invariants with validator-based admission.

These baselines isolate the effect of treating the protocol bundle as the governing artifact for generated software.

Ambiguity and the Natural Language Tax

To measure ambiguity, a study should generate multiple implementations from the same design intent under SDD, TDD, and PDD conditions and compare them along three axes:

- **Structural divergence:** differences in interface shape, field interpretation, schema compatibility, and version behavior;
- **Behavioral divergence:** differences in outputs, error semantics, determinism, and property satisfaction;
- **Operational divergence:** differences in external dependencies, side effects, latency, and resource usage.

These measurements make the *Natural Language Tax* observable: interpretation cost appears as structural divergence, reconciliation cost as behavioral divergence, and maintenance cost as operational drift. Evidence would support PDD if protocol-visible divergence is lower than under SDD or TDD, and would weaken it if divergence remains comparable or greater despite the added protocol-authoring burden.

Regeneration Reliability

Future studies should test regeneration by repeatedly generating implementations for the same protocol using:

- different generative models;
- multiple prompting strategies;
- varied sampling temperatures;
- alternative programming languages and runtime frameworks.

The relevant metrics are:

- **validation pass rate,**
- **number of attempts to first successful admission,**
- **time to successful admission,**
- **rate of protocol-level substitutability among admitted implementations.**

Evidence would support PDD if the protocol remains a stable target across generation methods and admitted implementations remain substitutable under the protocol's observation model. It would weaken PDD if small changes in model, prompt, or language commonly produce candidates that cannot satisfy the protocol or require extensive manual repair.

Protocol-Level Substitutability

Substitutability should be evaluated by placing independently generated implementations of the same protocol behind the same interface boundary and measuring:

- functional outputs and error semantics;
- client-visible timing behavior within admissible operational envelopes;
- downstream compatibility with dependent services;
- protocol-visible side effects and authority usage.

This evaluation asks whether a protocol-respecting client can replace one admitted implementation with another without relying on behavior outside the protocol. Failures indicate underspecified protocols, unsound validators, or clients coupled to implementation accidents.

Validation Cost

The Validator Loop adds work relative to conventional build-and-test pipelines. An empirical assessment should measure:

- structural validation time;
- behavioral validation time, including property-based and regression checks;
- operational validation overhead introduced by sandboxing and instrumentation;
- evidence generation and recording latency;
- runtime attestation overhead for live monitoring;
- total time from candidate generation to admission decision.

The central question is whether added cost is proportionate to gains in governance, substitutability, and auditability. A negative result occurs if validation overhead dominates without improving admission quality.

Governance Efficacy

To evaluate governance directly, future studies should include non-compliant candidates that violate operational or provenance constraints:

- unauthorized network access;
- hidden temporary file writes;
- excessive database calls;
- use of unapproved dependencies;
- latency-budget violations;
- missing or malformed provenance metadata;
- production-only races, memory leaks, dependency drift, and workload spikes.

The main metric is the fraction of violations detected and blocked by the Validator Loop before admission and by the Runtime Verification Layer after deployment. PDD is strengthened if operational and provenance violations are systematically rejected before admission and post-deployment anomalies produce signed evidence and targeted repair contexts. It is weakened if non-compliant candidates pass validation at rates comparable to conventional test-only pipelines or if deployed violations are visible in telemetry but absent from the Dynamic Evidence Ledger.

Evidence and Reproducibility

For every admitted implementation, the Discovery Log, Evidence Chain, and Dynamic Evidence Ledger should be assessed by whether they allow an auditor or downstream system to:

- reconstruct the exact protocol version and dependency closure;
- verify artifact hashes and provenance metadata;
- replay validation decisions deterministically when the required inputs are preserved;
- trace the admission decision from protocol to implementation to deployment artifact;

- inspect runtime attestation blocks, violation evidence, and remediation outcomes.

This dimension asks whether PDD produces usable admission and runtime evidence rather than only validation reports. Strong evidence would show decisions and runtime observations that are inspectable and linked to the governing protocol; weak evidence would show an evidence record too incomplete, expensive, or fragile to support audit.

Validation and Falsification Criteria

The empirical agenda can be summarized as a set of falsifiable expectations:

1. PDD should reduce protocol-visible ambiguity relative to SDD and TDD baselines.
2. Regenerated implementations should repeatedly reach admission under the same protocol.
3. Independently generated implementations should be substitutable for protocol-respecting clients.
4. Validation overhead should be measurable and justified by governance benefits.
5. Operational and provenance violations should be detected before admission.
6. Runtime verification should detect anomalies that appear after deployment and produce signed evidence blocks.
7. Evidence artifacts should support reproducible admission decisions, runtime attestation, and retrospective audit.

Failure on any dimension would be informative: the protocol may be too weak, the validator unsound, the evidence insufficient, or authoring costs too high for the target system class.

Summary

This agenda specifies how PDD should be tested once implemented in concrete toolchains. To serve as authoritative development artifacts, protocols must measurably stabilize generated software, bound operational authority, and produce evidence for admission, runtime attestation, regeneration, substitution, remediation, and audit.

Related Work

Protocol-Driven Development (PDD) draws on formal verification, runtime verification, executable testing, declarative infrastructure, software supply-chain security, and automated software engineering. Its contribution is not a standalone verification primitive, but a development model in which protocols are the enduring artifact and implementations are admitted and continuously attested through validation evidence.

Table 2 summarizes this positioning. The adjacent approaches are complementary; PDD differs by making a protocol bundle the primary artifact that jointly governs structure, behavior, operational authority, evidence-producing admission, and runtime attestation.

The distinction is artifact scope rather than replacement. PDD uses schemas, tests, policies, and provenance mechanisms as components of protocol-governed admission, while treating the protocol bundle as the durable admission artifact.

Formal Verification, Testing, and Executable Correctness

The formal foundations of PDD lie in formal methods. Hoare logic, model checking, TLA+, and Alloy show that correctness can be expressed through machine-checkable assertions, state-space exploration, invariants, temporal properties, and relational constraints (Hoare 1969; Clarke, Grumberg, and Peled 1999; Lamport 2002; Jackson 2002). These traditions support a premise of PDD: some correctness boundaries are more stable than the code that realizes them.

Test-Driven Development made executable artifacts central to engineering practice (Beck 2003); property-based testing generalized examples into laws over generated inputs (Claessen and Hughes 2000).

Runtime Verification and Online Monitoring

Runtime verification monitors executions against formal or executable properties after deployment (Leucker and Schallhart 2009; Havelund and Roşu 2004). PDD adopts this idea for generated software governance but changes the artifact boundary: the runtime monitor is not a standalone specification divorced from development. It evaluates observations against the same protocol bundle that governed admission, appends signed results to a Dynamic Evidence Ledger, and can feed violation evidence back into the generation and validation loop.

Specifications, Interfaces, and Declarative Artifacts

Traditional software engineering has long relied on specifications ranging from prose requirements to machine-readable interfaces. Systems such as OpenAPI, JSON Schema, and Protocol Buffers stabilize communication boundaries and enable structural automation (OpenAPI Initiative 2021; JSON Schema 2022; Google 2008), but they typically capture shape and compatibility rather than full behavioral and operational semantics.

Policy-as-Code, Zero Trust, and Supply-Chain Provenance

The paper also draws on policy-oriented governance. Zero-trust architecture replaces implicit trust with explicit verification (Rose et al. 2020), and policy-as-code systems such as Open Policy Agent decouple authorization logic from application code (Styra 2016). These approaches show the value of declarative governance evaluated independently of the governed artifact.

Supply-chain frameworks such as in-toto and SLSA extend this principle to provenance, grounding trust in attestations, traceability, and build metadata (Torres-Arias et al. 2019; Open Source Security Foundation (OpenSSF) 2021).

Automated Software Engineering and Program Synthesis

The motivating context for PDD is the rise of code-generating language models and automated synthesis engines. The “Software 2.0” framing recast parts of software construction as search over program space (Karpathy 2017); recent systems synthesize programs from prompts, improve productivity on

Table 2: Protocol-Driven Development compared with adjacent approaches.

Approach	Structure	Behavior	Operational authority	Evidence / provenance	Primary artifact
Interface schemas / API descriptions	Schemas, signatures	Mostly external	Mostly external	Version metadata	Interface contract
TDD / property-based testing	Indirect via fixtures	Examples, properties	Usually external	Test results	Test suite
Policy-as-code / sandbox policies	Policy inputs	Policy predicates	Capabilities, resources	Decision logs	Policy rule set
Supply-chain provenance / attestations	Artifact metadata	Not specified	Not specified	Attestations, lineage	Provenance record
Runtime verification	Instrumented events	Runtime properties	Monitor actions	Execution traces	Monitor specification
Protocol-Driven Development (PDD)	Typed handshakes	Behavioral invariants	Capability manifest	Evidence Ledger	Protocol bundle

selected tasks, and tackle repository-level software engineering benchmarks (Chen et al. 2021; Peng et al. 2023; Yang et al. 2024; Jimenez et al. 2024; Wu 2024).

Much of this literature focuses on generation capability, repair ability, and task completion. Here the emphasis is admission and governance.

Relationship to Runtime Governance

PDD is related to runtime governance for probabilistic systems, where model output is treated as a proposal, authority is placed in explicit constraints, and consequential admissions preserve evidence. Sovereign Agentic Loops and OpenKedge are runtime examples: SAL separates reasoning from execution through a control boundary (He and Yu 2026b), while OpenKedge evaluates runtime operations through policy, contracts, and evidence records (He and Yu 2026a).

PDD is the development-time counterpart to this pattern and extends it with continuous attestation. Runtime governance asks whether an action may execute; PDD asks whether a generated implementation is admissible before it becomes part of the system and whether its deployed behavior remains within protocol bounds. In both cases, a proposal is evaluated against explicit constraints and acceptance is accompanied by evidence.

This relationship is contextual rather than necessary. PDD stands on its own protocol model: structural invariants, behavioral invariants, operational authority, the Validator Loop, and Evidence Chains. Testing asks whether code behaves as expected in selected cases; PDD asks whether generated code has crossed a development boundary in which typed handshakes, behavioral laws, operational capabilities, validation evidence, and provenance align.

Synthesis: Positioning of PDD

Prior work provides mature mechanisms for interface definition, formal specification, policy enforcement, provenance, and code generation, but these mechanisms usually operate in separate lifecycle stages. PDD combines them into a development-time admission model: it generalizes interfaces from structural shape to admissibility boundaries, incorpo-

rates evidence into admission, and treats generated implementations as candidates rather than authorities.

Discussion and Future Work

Protocol-Driven Development (PDD) reframes software engineering around a shift in durable artifacts: implementation code is expected to change frequently, while protocols define admissible behavior. We consider implications, limitations, and directions for future research.

From Programming to Protocol Engineering

Under PDD, design effort shifts from writing a single implementation to authoring protocols that admit many possible implementations. Protocol authoring becomes a primary engineering activity: architects define typed handshakes, behavioral assertions, capability manifests, and validator requirements. The implementation becomes a replaceable witness of these constraints.

Protocol Reuse and Protocol Registries

A natural next step is reusable protocol registries. Just as package repositories support library reuse, protocol registries would support reuse of interface contracts, behavioral constraints, and capability boundaries for recurring patterns such as idempotent APIs, financial handlers, or audit-logging services.

Regenerable Systems

Repeated regeneration is possible in response to improved generators, dependency vulnerabilities, language migration, or runtime violations. The admission condition remains unchanged: regenerated implementations must satisfy the governing protocol and produce valid evidence before replacing earlier realizations. Runtime evidence can make regeneration more targeted by identifying the violated invariant, input class, dependency state, or resource envelope that triggered repair.

Continuous Attestation

Static evidence should not be treated as permanent proof of production behavior. Dynamic Evidence Ledgers extend auditability across the operational lifetime of a component by recording signed runtime observations, invariant checks, violations, and remediation outcomes. This shifts PDD from a deployment gate to a continuous governance loop: admission remains necessary, but deployed implementations continue to be measured against the monitorable projection of the protocol.

Protocol Inference and Synthesis

We assume protocols are authored by architects or human-machine collaborations. Future systems could infer invariants from existing code, telemetry, Discovery Logs, or regulatory documents, but machine-synthesized protocols raise open questions about validator trust, protocol quality, and verification of generated constraints.

Integration with Stronger Formal Methods

PDD is compatible with verification techniques ranging from property testing to formal proof. The Validator Loop can incorporate theorem provers, SMT solvers, model checkers, and certified compilers as mechanisms for making formal verification an operational component of software admission.

Economic Implications

Automated program synthesis makes candidate implementations cheaper while trustworthy specification and governance remain difficult. Under PDD, durable engineering assets include protocol libraries, validator implementations, and domain-specific capability policies, not proprietary source code alone.

Limitations

Protocol-Driven Development does not eliminate engineering failure. A poorly authored protocol may miss intent; an unsound validator may admit non-compliant implementations; runtime monitors may introduce overhead, false positives, blind spots, or privacy constraints; and some properties may be expensive, incomplete, or undecidable to verify. Runtime evidence can also be misleading if telemetry is sampled poorly, if sensitive observations are over-redacted, or if the monitorable projection omits the property that actually failed. Protocol authoring may be unjustified for very small, short-lived, or low-risk systems. PDD shifts difficulty from implementation authorship toward protocol quality, validator soundness, runtime monitor fidelity, and evidence integrity.

Relationship to Sovereign Engineering

Beyond the development model proposed here, PDD fits a broader vision of *sovereign engineering*: consequential transitions are mediated by explicit constraints and verifiable evidence. In this setting, decision authority belongs to a governing artifact or control boundary rather than to the probabilistic generator that produced a proposal.

OpenKedge and Sovereign Agentic Loops provide runtime examples of this pattern. PDD applies the same idea to

software construction, governing whether proposed implementations may enter the codebase and whether deployed implementations remain within their protocol envelope. The framework remains self-contained as a protocol model while allowing runtime systems to contribute continuous evidence.

Summary

Protocol-Driven Development defines a design space in which software systems are specified through protocols, re-generated under explicit constraints, and governed through validation and runtime evidence. Future work includes protocol registries, automated protocol synthesis, formal-verification integration, runtime attestation policies, and deployment studies.

Conclusion

Automated synthesis engines have changed the economics of software engineering. Candidate implementations are becoming cheaper to produce and easier to regenerate. Under these conditions, the primary challenge is no longer only to author code efficiently, but to define the boundaries within which generated code may be admitted, composed, and replaced.

We proposed **Protocol-Driven Development (PDD)**, a software engineering model in which the primary artifact is a machine-enforceable protocol rather than implementation code. We defined a protocol as a triplet of structural, behavioral, and operational invariants that together characterize the admissible implementation space of a software component. Under this formulation, implementations are transient realizations discovered through constrained search rather than permanent objects of engineering preservation.

We outlined the *Validator Loop* to separate protocol authoring, candidate generation, verification, and admission into distinct roles. Automated generators may explore the implementation space, but an artifact becomes admissible only when a validation engine establishes compliance and emits verifiable evidence. Discovery Logs and Evidence Chains make acceptance auditable and accountable; replay is supported when validator inputs are preserved.

We then extended this admission model into operation through Dynamic Evidence Ledgers and a Runtime Verification Layer. Runtime observations, invariant checks, violation reports, and remediation outcomes can be appended as signed evidence, allowing deployed implementations to remain accountable to the protocol after release. When runtime attestation fails, the violation becomes structured repair context, but any generated patch must still pass validation before replacement.

We further developed a theoretical foundation for PDD by modeling software construction as search over a protocol-defined implementation set and formalizing basic properties such as sound acceptance, protocol-level substitutability, and monotonic strengthening under protocol refinement. These results formalize the intuition that implementations satisfying the same protocol can vary internally while remaining interchangeable under the protocol's observation model.

We also presented a reference architecture, illustrative case studies, and an evaluation agenda outlining how PDD can be

applied and tested across functions, pipelines, and automated microservices. By combining ideas from formal methods, executable testing, runtime verification, policy-as-code, software provenance, and automated software engineering, PDD defines a governance model for software synthesis under low-cost implementation generation.

As implementation cost approaches zero, the enduring intellectual product of software engineering becomes the set of formal constraints that define admissible behavior, together with the evidence that those constraints were respected.

Protocol-Driven Development frames software construction and operation as constrained discovery with explicit evidence. It points toward systems in which reasoning, implementation, and execution are treated as proposals admitted through formal boundaries and verifiable evidence. Code remains changeable, but the protocol carries the durable authority to define which generated software artifacts are admissible and whether deployed behavior remains within bounds.

Appendix: Illustrative Minimal PDD Artifact Bundle

This appendix sketches an illustrative minimal PDD artifact bundle. The example is not prescriptive: concrete systems can use OpenAPI, Protocol Buffers, JSON Schema, Rego, TLA+, property-based testing frameworks, in-toto attestations, SLSA metadata, or other formats. The artifact blocks are schematic rather than implementation syntax. The protocol bundle collects structural, behavioral, and operational constraints into one versioned artifact, and the evidence record binds an admitted implementation back to that artifact.

Bundle Structure

A minimal bundle contains a manifest, invariant files, validator requirements, and an evidence namespace. One directory layout is:

```
fraud-score.protocol/  
  protocol.yaml  
  structural/  
    request-response.schema.yaml  
  behavioral/  
    scoring.properties.yaml  
  operational/  
    capabilities.yaml  
  validators/  
    validator-set.yaml  
  evidence/  
    admission-record.json  
    runtime-ledger.jsonl
```

The top-level manifest identifies the protocol, its version, and the invariant artifacts that define admission:

```
protocol_id: fraud-score  
version: 1.0.0  
component: risk.scoring.FraudScore  
invariants:  
  structural: structural/request-response.schema.yaml  
  behavioral: behavioral/scoring.properties.yaml  
  operational: operational/capabilities.yaml  
validators:  
  required_set: validators/validator-set.yaml  
evidence:  
  namespace: evidence/
```

Structural Invariant

The structural invariant fixes the typed handshake. A concrete implementation could encode it in JSON Schema, OpenAPI, Protocol Buffers, or another schema language.

```
request:  
  type: object  
  required: [transaction_id, account_id, amount_cents]  
  properties:  
    transaction_id: string  
    account_id: string  
    amount_cents: integer, minimum 0  
    merchant_country: ISO-3166 alpha-2 string  
response:  
  type: object  
  required: [transaction_id, risk_score, decision]  
  properties:  
    transaction_id: string  
    risk_score: number, range [0.0, 1.0]  
    decision: approve — review — decline  
errors: invalid_request — dependency_unavailable
```

Behavioral Invariant

The behavioral invariant records protocol-visible semantic properties. The pseudo-format below is intentionally neutral; a concrete validator could translate these entries into property-based tests, metamorphic checks, or formal assertions.

```
properties:  
  - name: deterministic_scoring  
    for_all: request  
    require: score(request) == score(request)  
  - name: score_range  
    for_all: request  
    require: 0.0 ≤ risk_score ≤ 1.0  
  - name: monotone_amount_risk  
    for_all: request_a, request_b  
    when: same fields except amount_cents  
    require: larger amount does not lower risk_score  
  - name: invalid_request_fails_closed  
    when: missing required field  
    require: error.kind == invalid_request
```

Operational Invariant

The operational invariant (often expressed as a capability manifest) constrains what the generated implementation is permitted to do while satisfying the structural and behavioral contract.

```
capabilities:  
  network:  
    outbound_allowlist: [feature-store.internal:443]  
    deny_other_outbound: true  
  filesystem:  
    read: []  
    write: []  
  dependencies:  
    allow: [risk-common, protocol-runtime]  
  resources:  
    max_latency_ms_p95: 75  
    max_memory_mb: 256  
    max_feature_store_calls_per_request: 1  
  secrets:  
    allow: [FEATURE_STORE_TOKEN]  
  background_work:  
    allowed: false
```

Evidence Object

An evidence object records the admission decision for one candidate implementation. The record is not the protocol itself; it is a signed link between a protocol version, a candidate artifact, validator execution, and observed validation results.

```
evidence_id: evd_2026_05_11_001
protocol:
  protocol_id: fraud-score
  version: 1.0.0
  bundle_digest: sha256:3b8f...
implementation:
  artifact_id: risk-scoring-service
  artifact_digest: sha256:91ac...
  language: python
  runtime: python-3.12
validators:
  - schema-conformance, version 0.4.2, result pass
  - property-check, version 0.9.1, result pass
    generated_cases: 5000
    counterexamples: 0
  - capability-monitor, version 0.3.0, result pass
    max_latency_ms.p95: 61
    network_violations: 0
    filesystem_writes: 0
decision: admit
issued_at: 2026-05-11T00:00:00Z
issuer: validation-engine.example
signature: sig:...
```

Runtime Ledger Block

After deployment, runtime attestation blocks can be appended to the evidence namespace. The example below records a monitorable operational violation without requiring the ledger to expose raw customer data.

```
ledger_block_id: evd_2026_05_11_runtime_0042
previous_block_digest: sha256:aa17...
protocol:
  protocol_id: fraud-score
  version: 1.0.0
implementation:
  artifact_digest: sha256:91ac...
  deployed_version: risk-scoring-service@2026.05.11
runtime_verifier:
  name: capability-monitor
  version: 0.3.0
interval:
  start: 2026-05-11T00:05:00Z
  end: 2026-05-11T00:06:00Z
attestation:
  decision: violation
  violated_invariant: max_feature_store_calls_per_request
  observed_value: 2
  allowed_value: 1
  trace_digest: sha256:f03c...
  raw_trace_location: access-controlled://traces/f03c...
action:
  runtime: route quarantined
  remediation_context: repairctx_2026_05_11_0042
signature: sig:...
```

The artifacts illustrate the separation central to PDD. The protocol bundle defines the admissibility boundary, the implementation is a candidate realization, the admission evidence records why that candidate entered the system, and runtime ledger blocks record whether deployed behavior remains

within the monitorable protocol boundary. Other systems can choose different concrete encodings while preserving the same artifact roles.

References

- Beck, K. 2003. *Test-Driven Development: By Example*. Addison-Wesley.
- Chen, M.; et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.
- Claessen, K.; and Hughes, J. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 268–279.
- Clarke, E. M.; Grumberg, O.; and Peled, D. A. 1999. *Model Checking*. MIT Press.
- Google. 2008. Protocol Buffers. <https://protobuf.dev/overview/>. Accessed: 2026-05-10.
- Havelund, K.; and Roşu, G. 2004. An Overview of the Runtime Verification Tool Java PathExplorer. *Formal Methods in System Design*, 24(2): 189–215.
- He, J.; and Yu, D. 2026a. OpenKedge: Runtime Governance for Sovereign Agentic Systems. Unpublished manuscript.
- He, J.; and Yu, D. 2026b. Sovereign Agentic Loops. Unpublished manuscript.
- Hoare, C. A. R. 1969. An Axiomatic Basis for Computer Programming. *Communications of the ACM*, 12(10): 576–580.
- Jackson, D. 2002. Alloy: A Lightweight Object Modelling Notation. *ACM Transactions on Software Engineering and Methodology*, 11(2): 256–290.
- Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. R. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *The Twelfth International Conference on Learning Representations*.
- JSON Schema. 2022. JSON Schema Draft 2020-12. <https://json-schema.org/draft/2020-12>. Accessed: 2026-05-10.
- Karpathy, A. 2017. Software 2.0. <https://karpathy.medium.com/software-2-0-a64152b37c35>. Accessed: 2026-05-10.
- Lamport, L. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.
- Leucker, M.; and Schallhart, C. 2009. A Brief Account of Runtime Verification. *The Journal of Logic and Algebraic Programming*, 78(5): 293–303.
- Open Source Security Foundation (OpenSSF). 2021. Supply-chain Levels for Software Artifacts. <https://slsa.dev/>. Accessed: 2026-05-10.
- OpenAPI Initiative. 2021. OpenAPI Specification Version 3.1.0. <https://spec.openapis.org/oas/v3.1.0.html>. Accessed: 2026-05-10.
- Peng, S.; Kalliamvakou, E.; Cihon, P.; and Demirer, M. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv:2302.06590.

Rose, S.; Borchert, O.; Mitchell, S.; and Connelly, S. 2020. Zero Trust Architecture. Technical Report NIST Special Publication 800-207, National Institute of Standards and Technology.

Styra. 2016. Open Policy Agent. <https://www.openpolicyagent.org/docs/policy-language>. Accessed: 2026-05-10.

Torres-Arias, S.; Afzali, H.; Kuppusamy, T. K.; Curtmola, R.; and Cappos, J. 2019. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *28th USENIX Security Symposium*, 1393–1410. USENIX Association.

Wu, S. 2024. Introducing Devin, the First AI Software Engineer. <https://www.cognition.ai/blog/introducing-devin>. Accessed: 2026-05-10.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K. R.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, volume 37.